



Home Assignment – QA Automation Engineer



Part 1: API Testing

Test the Restful Booker API

Base URL: <https://restful-booker.herokuapp.com>

This is a real API for a hotel booking system that allows full CRUD operations.

API Documentation: <https://restful-booker.herokuapp.com/apidoc/index.html>

Test the Graph QL

Select any available schema: Video, Ecommerce, Marketing

GraphQL Documentation: <https://hygraph.com/graphql-playground>

Required Test Scenarios:

Using **REST Assured** and **JUnit 5**:

1. Authentication:

None

POST /auth

Body: {"username": "admin", "password": "password123"}

- Get an auth token for subsequent requests

2. CRUD Operations:

- Create a new booking (POST /booking)
- Retrieve the booking by ID (GET /booking/{id})
- Update the booking (PUT /booking/{id})

- Delete the booking (DELETE /booking/{id})

3. GraphQL Positive

- Query a list with pagination/limit
- Query a single entity by ID.
- A query that uses GraphQL variables (not string interpolation).
- A query that uses a fragment or nested fields across types (e.g. movie → publishedBy → name)

4. GraphQL Negative

- Invalid ID (non-existent) — assert response shape (GraphQL typically returns HTTP 200 with data: null or an errors array; verify which).
- Malformed query (syntax error) — assert errors[].message and absence of data.
- Requesting a non-existent field — assert validation error.

Part 2: UI Testing

Test the DemoQA Website

URL: <https://demoqa.com>

This is a stable demo site specifically designed for automation practice.

Test These Sections:

Option A: Test Form Submission:

<https://demoqa.com/automation-practice-form>

- Fill the student registration form
- Upload a file
- Select a date from the date picker
- Choose from dropdowns
- Submit and verify the success modal

Option B: Test Web Tables: <https://demoqa.com/webtables>

- Add a new record
- Edit existing record
- Delete record
- Search functionality
- Sorting validation

UI Test Requirements:

- Use **Playwright**
- Implement Page Object Model
- Handle dynamic waits properly
- Take screenshots of failures

Core Tech Stack:

XML

```
<!-- Required Dependencies -->
- JUnit 5 (latest)
- REST Assured (API testing)
- Playwright for Java (UI testing)
- AssertJ (assertions)
- Jackson (JSON handling)

<!-- Bonus -->
- Allure (reporting)
- Lombok (reduce boilerplate)
```

Deliverables:

1. GitHub Repository with:

- Working test suite (both API and UI)
- Clear package structure
- **.gitignore** file
- **pom.xml** with all dependencies

2. README.md including:

```
None
# QA Automation Test Suite

## Prerequisites
- Java 11+
- Maven 3.6+
- Chrome browser

## How to Run
# Run all tests
mvn clean test

# Run only API tests
mvn test -Dgroups="api"

# Run only UI tests
mvn test -Dgroups="ui"

## Test Strategy
[Explain your approach, what you prioritized and why]

## Challenges & Solutions
[Any issues you encountered and how you solved them]

## What I Would Add With More Time
[Additional tests or improvements]
```

3. Test Report:

- Screenshot of test execution
- OR Surefire HTML report
- OR Allure report (bonus)

4. CI/CD (Bonus):

- A GitHub Actions workflow that runs tests



What We're Looking For (quality over quantity):

1. Test Design (20%)

- Meaningful test scenarios
- Good mix of positive/negative tests
- Clear test names and structure

2. **Code Quality (30%)**

- Clean, maintainable code
- Proper use of Page Object Model
- DRY principle (no duplication)
- Good assertions

3. **Framework Architecture (40%)**

- Logical project structure
- Reusable components
- Configuration management
- Base classes and helpers

4. **Documentation (10%)**

- Clear README
- Code comments where needed
- Meaningful commit messages

Minimum Requirements:

Must have:

- At least 3 API tests covering CRUD operations
- At least 5 API tests covering GraphQL
- At least 2 UI tests with Page Object Model
- Basic assertions using AssertJ
- README with setup instructions

Nice to have:

- Data-driven tests
- Custom waits/retry logic
- Allure reporting

- Parallel execution
- CI/CD setup

Important Notes:

- These are PUBLIC test services – be respectful, don't overload them
- Restful Booker resets data periodically, that's normal
- Focus on demonstrating best practices over test quantity
- If a service is temporarily down, document it and mock the scenario
- Commit frequently to show your development process

Getting Started:

Shell

Quick start

```
mvn archetype:generate -DgroupId=com.flamingo.qa \
  -DartifactId=qa-automation-assignment \
  -DarchetypeArtifactId=maven-archetype-quickstart \
  -DinteractiveMode=false
```

```
cd qa-automation-assignment
```

Add dependencies to pom.xml

Start with API tests first

Then move to UI tests

Questions? Reach out to Oleksandra. Good luck and happy testing! 🚀